

PetCare Veterinaria

Aplicación Android de Gestión Veterinaria

DOCUMENTACIÓN TÉCNICA

Natividad Valenzuela Valderrama · DAM 2025

1. ¿Qué es esta aplicación?

PetCare es una aplicación móvil para Android diseñada para gestionar el día a día de una clínica veterinaria a un nivel básico. Permite llevar un registro completo de los pacientes (mascotas), sus dueños (usuarios), las citas programadas y el historial médico de cada animal.

La app tiene dos tipos de usuario con permisos distintos:

- MASTER (veterinario): puede crear, leer, actualizar y eliminar pacientes y citas. En cuanto a los usuarios, solo puede leerlos, actualizarlos y eliminarlos.
- USER (dueño de mascota): solo puede leer sus propias mascotas, crear un usuario y contactar con la clínica.

2. Descripción de la arquitectura

La arquitectura elegida es la más sencilla posible para una app Android de este tamaño: arquitectura de una sola capa (también llamada arquitectura monolítica* simple). Cada pantalla (xml) tiene su propio archivo Java (.java) que se encarga tanto de mostrar los datos como de comunicarse con la base de datos.

** Monolítica: todo el código vive en el mismo sitio, sin dividirlo en capas separadas de "lógica de negocio", "datos", etc. Es perfecta para proyectos pequeños porque es más fácil de entender y mantener.*

¿Por qué esta arquitectura y no otra?

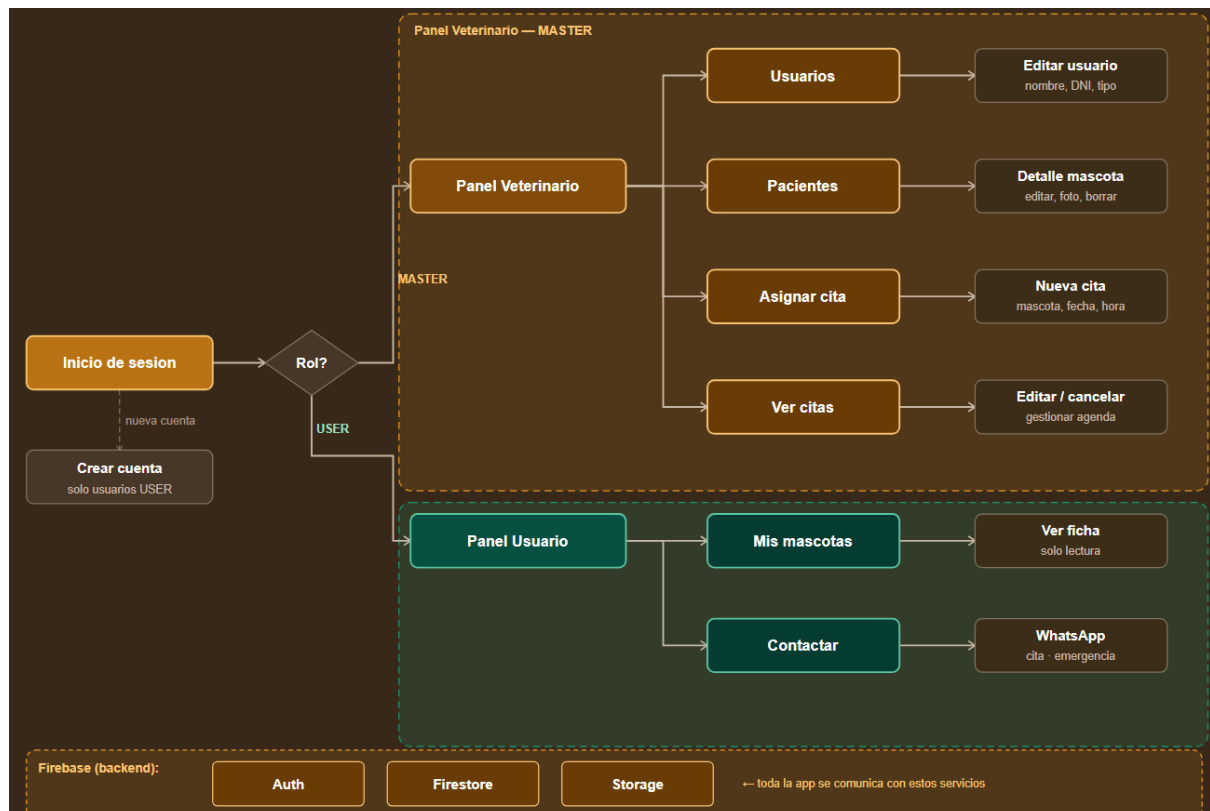
Para una app de estas dimensiones (destinada a una empresa pequeña, de unos 5 trabajadores aproximadamente), una arquitectura más compleja como MVVM* o Clean Architecture** habría añadido complejidad innecesaria sin aportar beneficio real. La regla general es: cuanto más pequeño el proyecto, más simple la arquitectura. No obstante, siempre se puede escalar en el futuro si la app crece, lo que haría que su arquitectura fuera replanteada en base a las necesidades concretas del momento.

** MVVM (Model-View-ViewModel): patrón de diseño que separa la pantalla (View), los datos (Model) y la lógica intermedia (ViewModel).*

***Patrón de diseño que organiza el código en capas concéntricas e independientes, cuyo objetivo principal es separar la lógica de negocio de los detalles técnicos como frameworks, bases de datos o interfaces de usuario*

Diagrama general del flujo

El flujo de la aplicación es lineal y sencillo:



El móvil se comunica directamente con los tres servicios de Firebase a través de internet (HTTPS). No hay servidor propio intermediario — Firebase actúa tanto de servidor como de base de datos.

Es decir, la aplicación instalada en el móvil se conecta por sí sola a Firebase para guardar datos, obtener información o gestionar usuarios. No necesita pasar antes por un servidor desarrollado por nosotros que haga de intermediario.

En este caso, Firebase se encarga directamente de recibir las peticiones de la aplicación y almacenar o devolver la información necesaria. De esta forma, la arquitectura es más sencilla porque el móvil y Firebase se comunican directamente entre sí a través de internet.

3. Tecnologías utilizadas

Lenguaje y entorno de desarrollo

Tecnología	Para qué se usa y por qué se eligió
Java	Lenguaje de programación principal. Se eligió porque es el lenguaje más maduro y documentado para Android nativo, con más recursos de aprendizaje disponibles que Kotlin.
Android Studio	Entorno de desarrollo oficial de Google para apps Android.

Android SDK 24+	Versión mínima de Android compatible. Cubre más del 95% de los dispositivos Android actuales.
Gradle (KTS)	Sistema de construcción del proyecto. Gestiona las dependencias (librerías externas) y compila la app. KTS es la versión moderna usando Kotlin en lugar de Groovy.

Backend y base de datos

Tecnología	Para qué se usa y por qué se eligió
Firebase Authentication	Gestiona el registro e inicio de sesión de usuarios. Se eligió porque es gratuito hasta cierto límite, muy seguro, y elimina la necesidad de construir un sistema de login propio.
Firebase Firestore	Base de datos principal en la nube. Almacena usuarios, mascotas y citas. Es una base de datos NoSQL*, lo que significa que los datos se guardan como documentos en lugar de tablas. Ideal para datos que pueden variar en estructura (no todas las mascotas tienen los mismos campos).
Firebase Storage	Almacenamiento de archivos en la nube, usado para guardar las fotos de las mascotas. Permite que las fotos estén disponibles desde cualquier dispositivo.
Firebase Analytics	Estadísticas básicas de uso de la app incluidas automáticamente con Firebase.

* NoSQL: tipo de base de datos que no usa tablas con filas y columnas como Excel, sino que guarda los datos en formato de documentos (parecido a archivos JSON). Es muy flexible porque cada documento puede tener campos diferentes.

Librerías externas

Librería	Para qué se usa
Glide 4.16.0	Carga y muestra imágenes desde internet de forma eficiente. Se usa para mostrar las fotos de mascotas descargadas desde Firebase Storage. Se eligió porque es la librería más usada para esto en Android y gestiona automáticamente la caché* de imágenes.
AndroidX AppCompat	Proporciona componentes de interfaz compatibles con versiones antiguas de Android.
Material Components	Librería de Google con componentes visuales modernos (botones, campos de texto, etc.).
ConstraintLayout	Sistema de diseño de pantallas que permite colocar elementos con precisión sin necesitar muchos contenedores anidados.

* Caché: almacenamiento temporal en el dispositivo. Glide guarda las imágenes descargadas para no tener que volver a descargarlas cada vez que se muestran.

Control de versiones

Herramienta	Para qué se usa
Git	Sistema de control de versiones. Permite guardar el historial de cambios del código y volver atrás si algo falla.
GitHub	Plataforma en la nube donde se aloja el repositorio Git. Repositorio público en github.com/ValValNat/Veterinaria .

4. Estructura del proyecto

La estructura sigue la organización estándar de cualquier proyecto Android Studio:



¿Por qué esta estructura?

Es la estructura por defecto de Android Studio y la más reconocida por cualquier desarrollador Android. Separar el código Java (lógica) de los recursos XML (diseño) es una convención universal en el desarrollo Android que facilita la lectura y el mantenimiento del proyecto.

Pantallas y su función

Archivo Java	Qué hace
Inicio.java	Login con email y contraseña. Incluye bloqueo de cuenta tras 3 intentos fallidos durante 5 minutos. Redirige a MASTER o USER según el tipo de cuenta.
CrearPass.java	Registro de nuevos usuarios. Valida formato de DNI español (8 dígitos + letra según tabla oficial), email válido, contraseña mínimo 6 caracteres y que no sea igual al email.
PanelVeterinario.java	Menú principal para el MASTER con acceso a Usuarios, Pacientes, Ver citas y Asignar cita.
VeterinarioListado.java	Lista de todos los usuarios con buscador. Permite editar datos, cambiar tipo, gestionar mascotas asignadas (crear nueva o asignar existente) y eliminar usuarios con cuenta atrás de 5 segundos.
PacientesActivity.java	Lista de todas las mascotas con buscador. Permite crear nuevas mascotas con formulario completo (microchip, raza dinámica, vacunas por especie, condiciones médicas).

DetalleMascota.java	Ficha completa de una mascota. El MASTER puede editarla y tomar fotos con la cámara que se guardan en Firebase Storage. El USER solo puede verla.
AsignarCitaActivity.java	Formulario para crear una cita: selección de mascota (con autocompletado del dueño), fecha con calendario, hora en franja de 9:00-17:00 cada 30 min, y motivo.
CitasActivity.java	Lista de citas ordenadas por fecha y agrupadas por día. Permite editar (fecha, hora, motivo) y cancelar cada cita.
Mascotas_usuario.java	Vista del USER: muestra sus mascotas y un botón para contactar con la clínica por WhatsApp (citas o emergencias).

5. Diagrama de la base de datos

La base de datos es Firebase Firestore, una base de datos NoSQL en la nube organizada en colecciones (equivalente a las tablas de una base de datos tradicional) y documentos (equivalente a las filas).

Colecciones en Firestore

La base de datos tiene tres colecciones principales:

Colección: users

Campo	Descripción y tipo de dato
email	Correo electrónico del usuario. Texto (String).
nombre	Nombre completo. Texto (String).
dni	DNI español validado. Texto (String).
telefono	Teléfono con prefijo internacional. Texto (String).
tipo	Rol del usuario: 'USER' o 'MASTER'. Texto (String).

La ID del documento coincide con el UID* que genera Firebase Authentication al registrar al usuario.

** UID (User ID): identificador único que Firebase asigna automáticamente a cada usuario al registrarse. Es como el DNI dentro del sistema Firebase.*

Colección: mascotas

Campo	Descripción y tipo de dato
nombre	Nombre de la mascota. Texto (String).
especie	Perro o Gato. Texto (String).
raza	Raza de la mascota (libre o de la lista). Texto (String).
edad	Edad en texto (ej. '3 años'). Texto (String).
color	Color del pelaje. Texto (String).
microchip	Si tiene microchip. Booleano (true/false).
vacunas	Lista de vacunas administradas. Array de textos (List<String>).
condiciones	Lista de condiciones médicas. Array de textos (List<String>).
ownerId	UID del dueño (referencia al documento en 'users'). Texto (String).

Colección: citas

Campo	Descripción y tipo de dato
pacienteNombre	Nombre de la mascota citada. Texto (String).

mascotald	ID del documento de la mascota. Texto (String).
clienteEmail	Email del dueño. Texto (String).
fecha	Fecha en formato d/M/yyyy. Texto (String).
hora	Hora en formato HH:mm. Texto (String).
motivo	Motivo de la consulta. Texto (String).

Almacenamiento de fotos (Firebase Storage)

Las fotos de las mascotas se guardan en Firebase Storage con la siguiente estructura de rutas:

```

mascotas/{mascotald}/foto.jpg

```

Donde {mascotald} es la ID del documento de la mascota en Firestore. Esto permite vincular cada foto con su mascota de forma directa.

Relaciones entre colecciones

Firestore no tiene relaciones automáticas como una base de datos relacional*, pero se simulan usando el campo ownerId en mascotas, que apunta al UID del usuario dueño, y el campo mascotald en citas, que apunta a la mascota correspondiente.

** Base de datos relacional: tipo clásico de base de datos (como MySQL o PostgreSQL) donde las tablas se relacionan entre sí con claves foráneas. Firestore no funciona así, pero se pueden simular estas relaciones manualmente.*

Reglas de seguridad de Firestore

Las reglas de seguridad controlan quién puede leer y escribir qué datos:

- Colección users: cualquier usuario autenticado puede leer. Solo puede crear su propio documento. Solo puede modificar o borrar el MASTER o el propio usuario.
- Colección mascotas: cualquier usuario autenticado puede leer y escribir (necesario para asignar mascotas desde distintas pantallas).
- Colección citas: solo el MASTER puede leer y escribir.

6. Explicación de los CRUD

CRUD son las cuatro operaciones básicas que se pueden hacer con datos: Create (Crear), Read (Leer), Update (Actualizar) y Delete (Borrar). A continuación se detalla cómo se implementa cada operación en la aplicación.

Usuarios

Operación	Dónde ocurre	Cómo funciona
CREATE	CrearPass.java	El usuario rellena el formulario de registro. Se crea una cuenta en Firebase Auth y a continuación se guarda el documento en la colección 'users' con los datos del formulario.
READ	VeterinarioListado.java	Al abrir la pantalla de usuarios, se hace una consulta a la colección 'users'. El buscador filtra en tiempo real sobre los resultados ya cargados.
UPDATE	VeterinarioListado.java	El popup de edición usa set() con merge:true, que actualiza solo los campos enviados sin borrar el resto del documento.
DELETE	VeterinarioListado.java	Botón 🗑️ con confirmación y cuenta atrás de 5 segundos antes de ejecutar delete() en el documento de Firestore.

Mascotas

Operación	Dónde ocurre	Cómo funciona
CREATE	PacientesActivity.java / VeterinarioListado.java	Formulario completo con validación del nombre obligatorio. Se llama a add() en la colección 'mascotas'. Si se indica el email del dueño, se busca primero su UID en 'users' y se asigna como ownerId.
READ	PacientesActivity.java / Mascotas_usuario.java	PacientesActivity carga todas las mascotas. Mascotas_usuario filtra por whereEqualTo('ownerId', uid) para mostrar solo las del usuario en sesión.

UPDATE	DetalleMascota.java	Popup completo de edición con todos los campos. Usa set() con merge:true para no sobrescribir campos no incluidos.
DELETE	DetalleMascota.java / VeterinarioListado.java	Confirmación de borrado. Al eliminar, también se borra la foto asociada en Firebase Storage con storageRef.delete().

Citas

Operación	Dónde ocurre	Cómo funciona
CREATE	AsignarCitaActivity.java	El MASTER selecciona una mascota de un desplegable cargado desde Firestore. Al seleccionarla, se autocompleta el email del dueño. Se elige fecha (DatePicker), hora (desplegable 9:00-17:00) y motivo.
READ	CitasActivity.java	Se cargan todas las citas de la colección 'citas', se ordenan por fecha+hora y se agrupan visualmente por fecha mostrando una cabecera por día.
UPDATE	CitasActivity.java	Popup de edición con fecha (DatePicker), hora (Spinner) y motivo. Usa update() directamente sobre el documento.
DELETE	CitasActivity.java	Botón 'Cancelar cita' con confirmación. Usa delete() sobre el documento de la cita.

7. Seguridad y autenticación

Este es un punto que merece mención especial porque la aplicación implementa varias capas de seguridad:

Autenticación (inicio de sesión)

- Se usa Firebase Authentication con email y contraseña.
- Si el usuario falla 3 veces seguidas, la cuenta se bloquea durante 5 minutos. Esto se gestiona con `SharedPreferences`* en el dispositivo para evitar ataques de fuerza bruta*.

* *SharedPreferences*: pequeño almacenamiento clave-valor en el dispositivo Android, como una memoria persistente para guardar ajustes simples.

* *Ataque de fuerza bruta*: técnica de hackeo que consiste en probar miles de contraseñas hasta dar con la correcta.

Roles de usuario

- Al iniciar sesión, la app consulta el campo 'tipo' del usuario en Firestore.
- Si es MASTER, va al Panel Veterinario con acceso completo.
- Si es USER, va a la pantalla de Mis Mascotas con acceso solo lectura.
- En `DetalleMascota`, la detección del rol se hace en tiempo real consultando Firestore, por lo que los botones de editar/borrar solo aparecen si el usuario es efectivamente MASTER.

Seguridad en la base de datos

- Las reglas de Firestore aseguran que un usuario normal no puede modificar datos de otro usuario aunque lo intente directamente.
- Las fotos en Storage solo son accesibles para usuarios autenticados.

Validaciones en formularios

- DNI español: valida los 8 dígitos + letra usando la tabla oficial (TRWAGMYFPDXBNJZSQVHLCKE).
- Email: usa `Patterns.EMAIL_ADDRESS` de Android para verificar el formato.
- Contraseña: mínimo 6 caracteres y no puede ser igual al email.

8. Funcionalidades destacadas

Gestión completa de mascotas

La ficha de cada mascota incluye campos específicos del sector veterinario: especie (perro/gato), raza (con lista dinámica según especie), edad, color del pelaje, microchip (sí/no), lista dinámica de condiciones médicas y lista de vacunas con checkboxes específicas por especie (vacunas de perro o de gato). Todos estos campos se cargan y guardan directamente en Firestore.

Cámara integrada para fotos

El MASTER puede fotografiar directamente desde la app usando la cámara del dispositivo. La foto se sube a Firebase Storage en la ruta `mascotas/{id}/foto.jpg`. Si la mascota ya tiene

foto, se muestra automáticamente al abrir la ficha. Si no la tiene, se muestra un icono de cámara para añadirla. Al eliminar una mascota, su foto también se borra de Storage.

Contacto por WhatsApp

Los usuarios de tipo USER pueden contactar con la clínica pulsando un botón que abre WhatsApp directamente con un mensaje predefinido, distinguiendo entre solicitar una cita (+34 722610920) y comunicar una emergencia (+34 698745348).

Pie legal en la pantalla del usuario

La pantalla de Mis Mascotas incluye un pie de página con la información legal básica (nombre de empresa, CIF, dirección, referencia al RGPD y email de privacidad), cumpliendo con los requisitos mínimos de las apps publicadas en España.

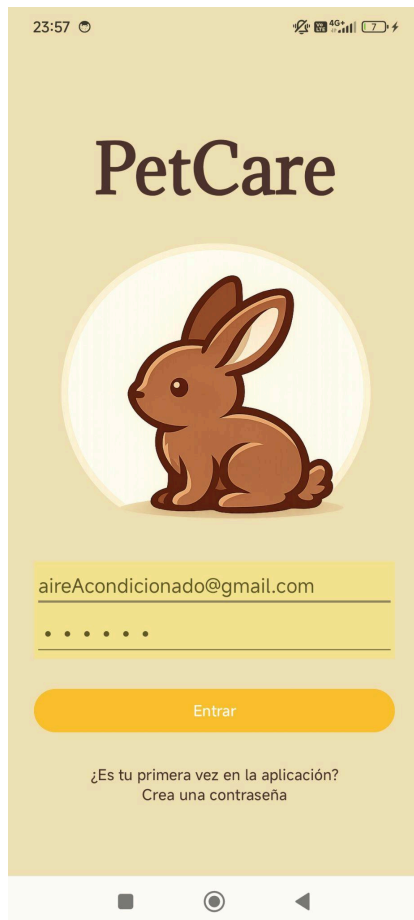
Buscador en tiempo real

Las pantallas de Usuarios y Pacientes incluyen un campo de búsqueda que filtra los resultados en tiempo real mientras el usuario escribe, sin necesidad de pulsar ningún botón.

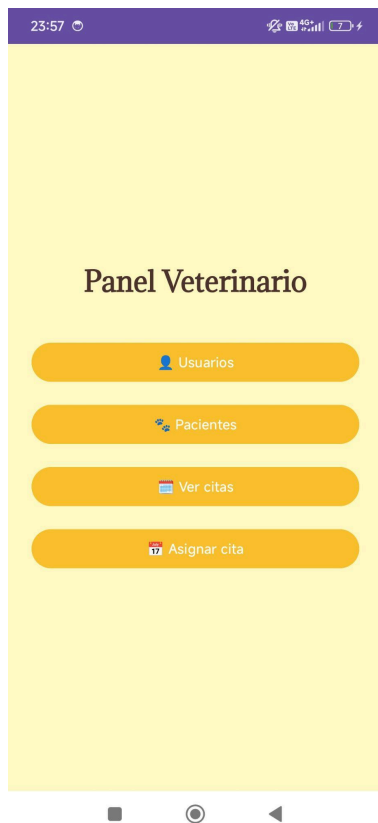
9. Capturas de pruebas

a) MASTER

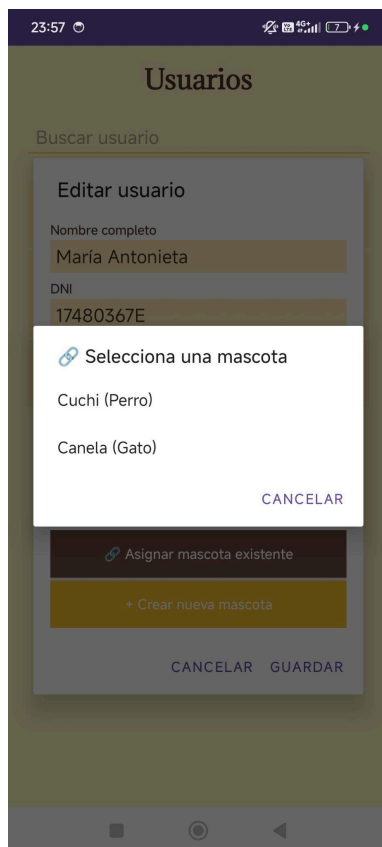
1. Inicio de sesión

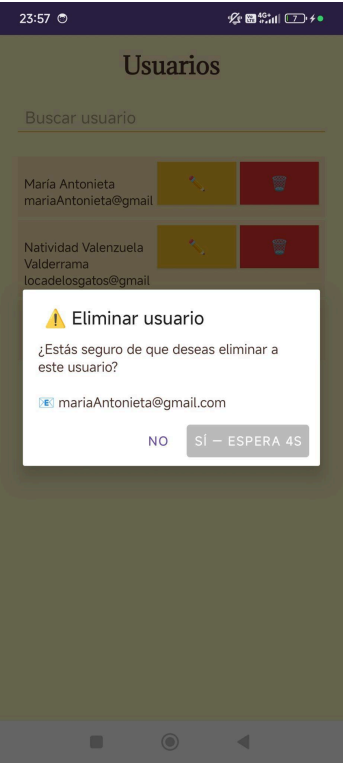
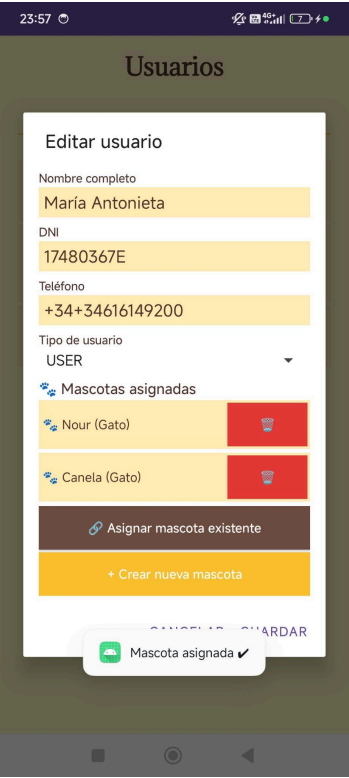


2. Menú

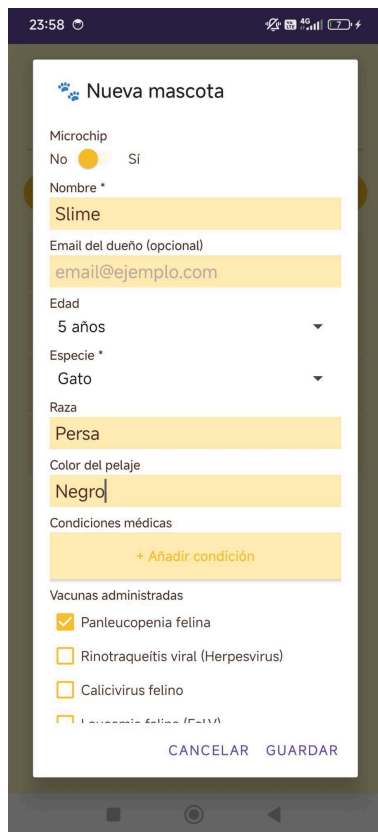
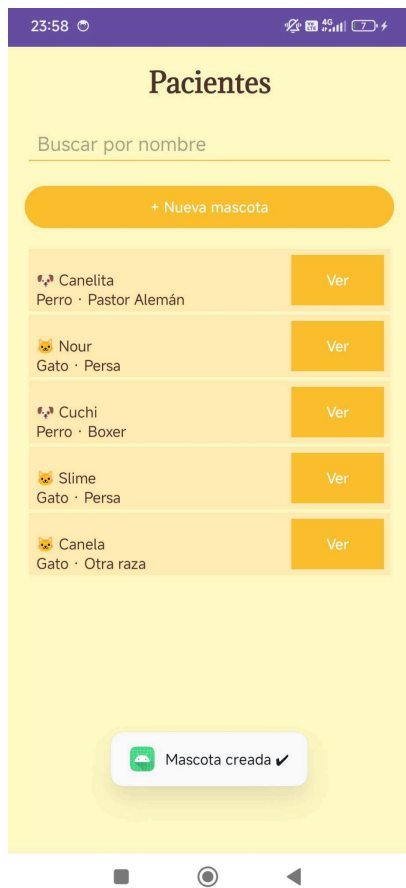


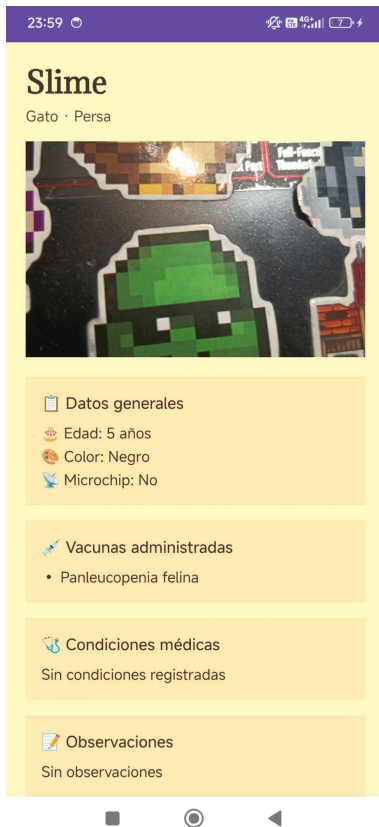
3. Gestión de usuarios



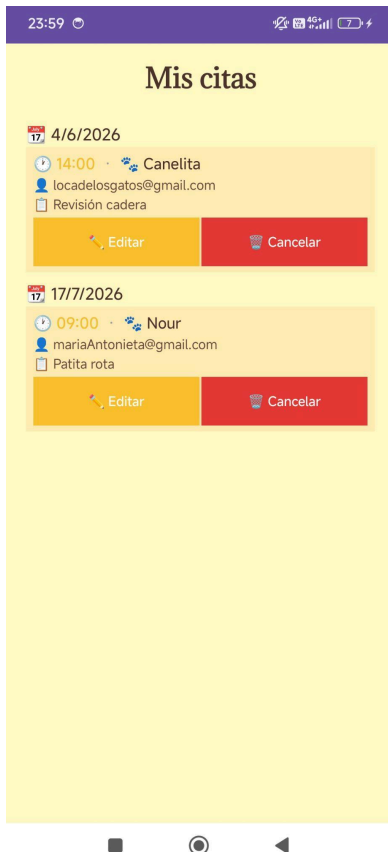


4. Gestión de pacientes (mascotas)





5. Consultar citas



6. Gestión de citas

23:59

Asignar Cita

Paciente (mascota)

Nour

Cliente (dueño)

8MpimKosX8TJ9JK18pydXI5Mx2q1

Fecha

16/6/2026

Seleccionar fecha

Hora

12:00

Motivo

Cura

Guardar cita

0:00

Mis citas

17 4/6/2026

16:00 · Canelita

locadelosgatos@gmail.com

Revisión cadera

Editar

Cancelar

17 16/6/2026

12:00 · Nour

8MpimKosX8TJ9JK18pydXI5Mx2q1

Cura

Editar

Cancelar

17 17/7/2026

09:00 · Nour

mariaAntonieta@gmail.com

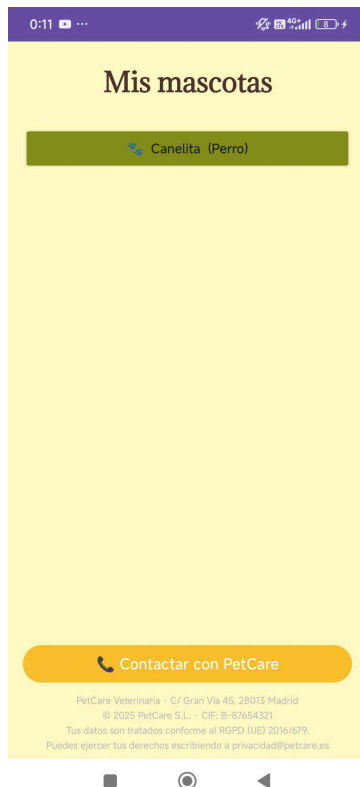
Patita rota

Editar

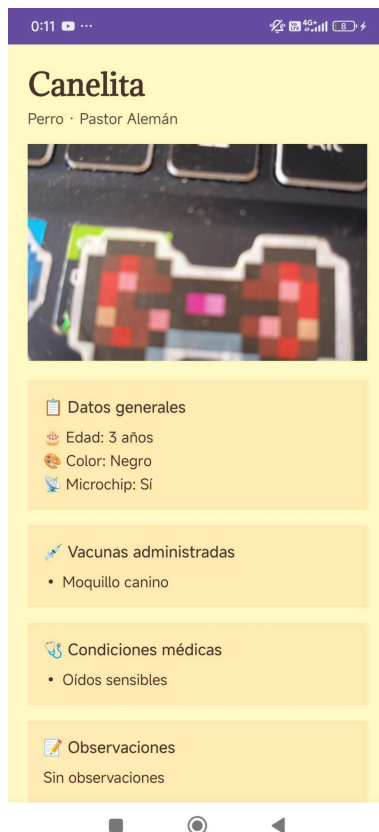
Cancelar

b) USER

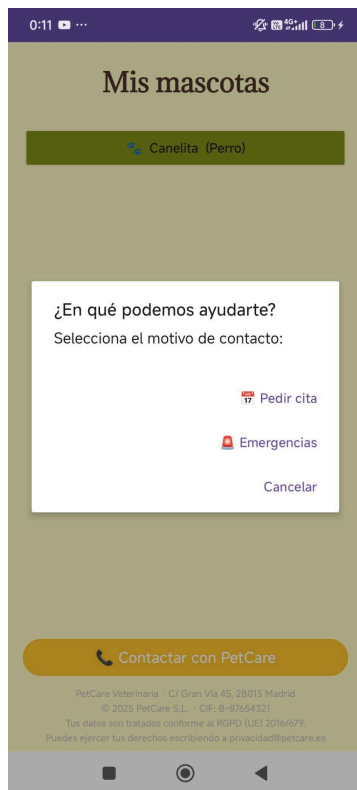
1. Menú principal tras iniciar sesión



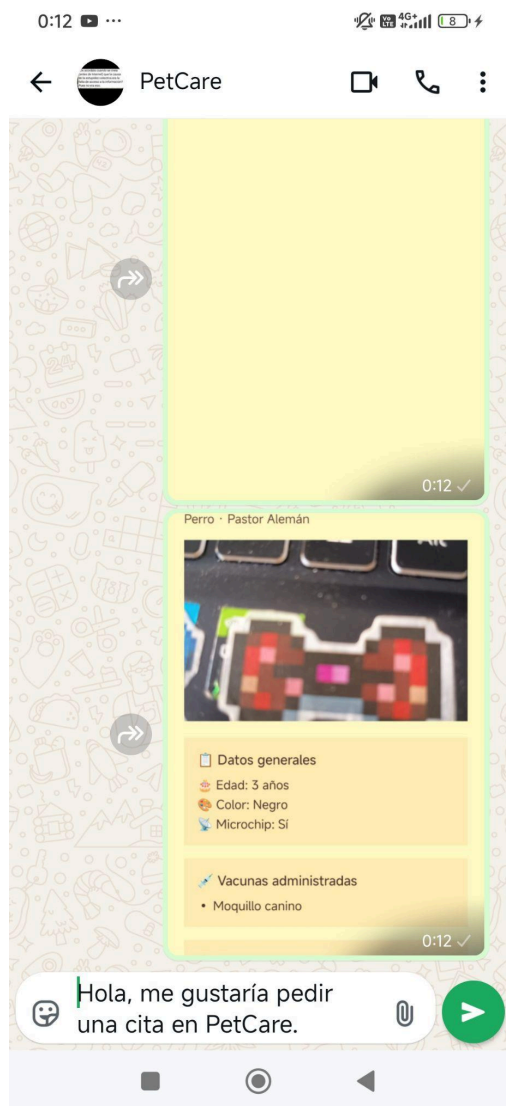
2. Ficha de mascota



3. Botón para contactar



4. WhatsApp



10. Explicación del despliegue

La aplicación está en formato apk para ser descargada en Android.

Infraestructura en producción

El backend (base de datos, autenticación y almacenamiento) está alojado en los servidores de Google a través de Firebase, plan Blaze (pago por uso). Con el volumen de datos actual de una clínica veterinaria pequeña, el coste mensual estimado es de 0€, ya que no se superan los límites del nivel gratuito incluido en el plan Blaze.

11. Control de versiones

El proyecto usa Git como sistema de control de versiones y GitHub como repositorio remoto. El repositorio es privado y está disponible en github.com/ValValNat/Veterinaria.

El flujo de trabajo es sencillo: rama main única con commits incrementales que describen cada cambio implementado. Se usa un archivo `.gitignore` para excluir carpetas de build, archivos de configuración local y credenciales sensibles.

12. Posibles mejoras futuras

- Notificaciones push* para recordar citas al dueño de la mascota.
- Histórico de visitas por mascota con fecha y notas del veterinario.
- Exportación de la ficha de una mascota a PDF.
- Versión iOS de la aplicación.
- Panel web para el veterinario accesible desde un ordenador.
- Sistema de valoraciones o comentarios de los clientes.

** Notificaciones push: mensajes que la app envía al móvil aunque esté cerrada, como los avisos de WhatsApp.*
